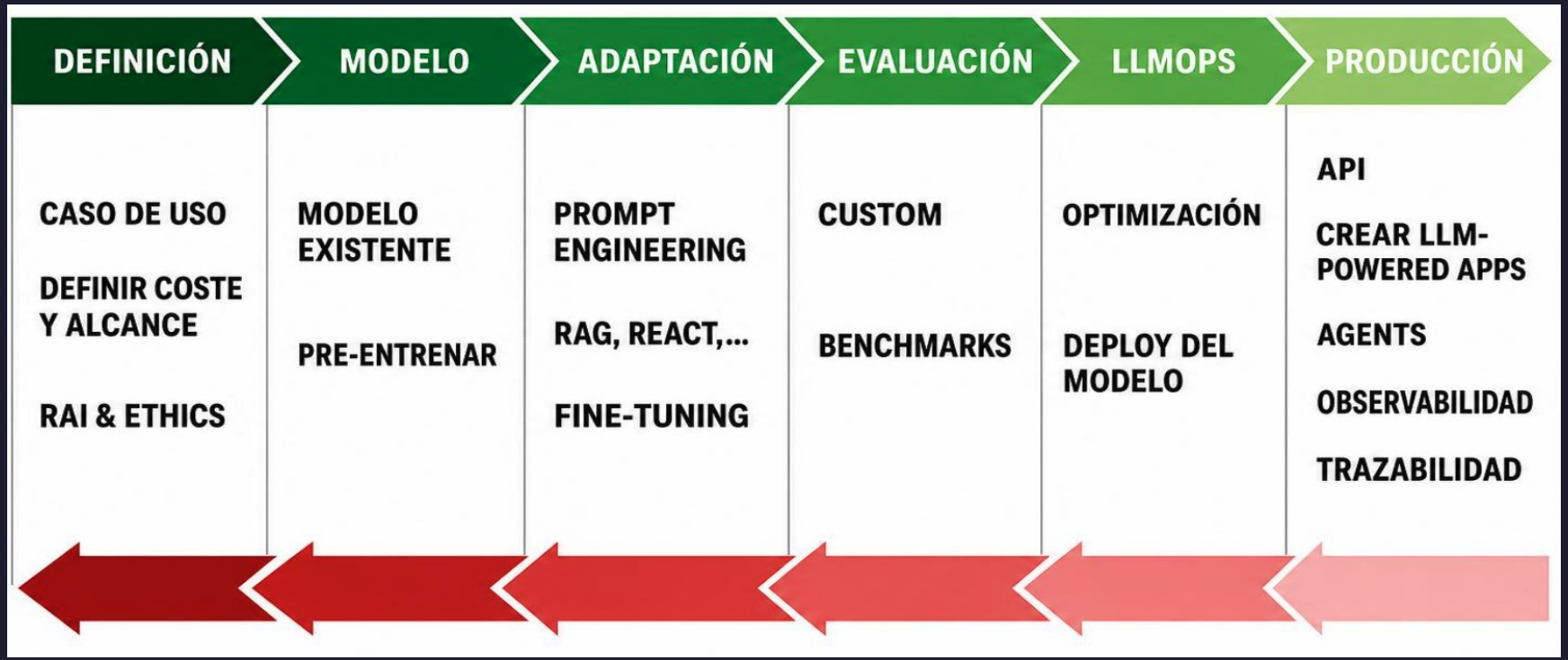
keep coding

AI ENGINEERING

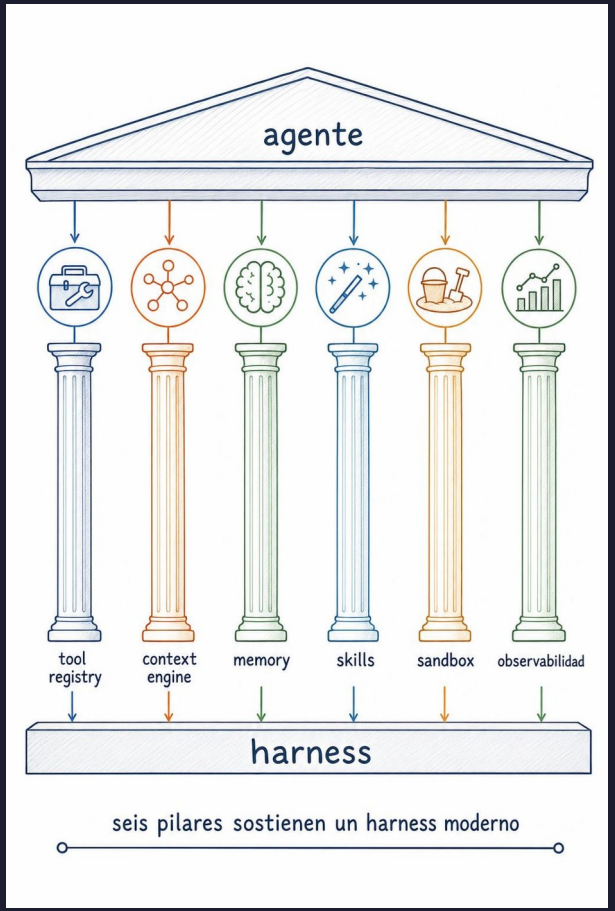
Harness Engineering


DEFINICIÓN
MODELO
ADAPTACIÓN
EVALUACIÓN
LLMOPS
PRODUCCIÓN
CASO DE USO
DEFINIR COSTE Y ALCANCE
RAI & ETHICS
MODELO EXISTENTE
PRE-ENTRENAR
PROMPT ENGINEERING
RAG, REACT,...
FINE-TUNING
CUSTOM
BENCHMARKS
OPTIMIZACIÓN
DEPLOY DEL MODELO
API
CREAR LLM-POWERED APPS
AGENTS
OBSERVABILIDAD
TRAZABILIDAD

keep coding

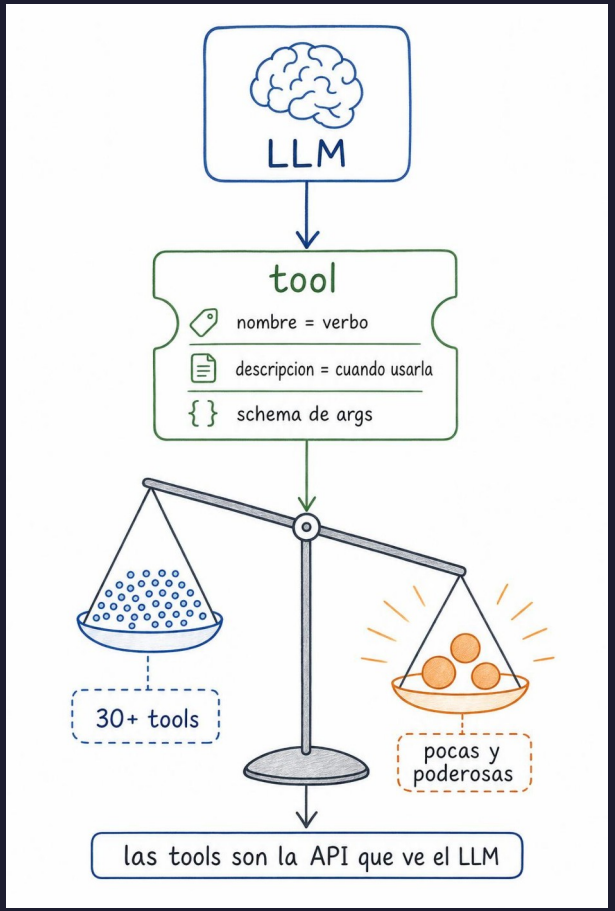
Anatomía de un harness · 6 pilares

- Tool registry · funciones expuestas al modelo con name y schema
- Context engine · qué cargar al system prompt en cada turno
- Memory · file-system, vector o graph · estado persistente
- Skills y sandbox · capacidades descubribles y aislamiento del host
- Observability · traces, replays y scores cierran los 6 pilares



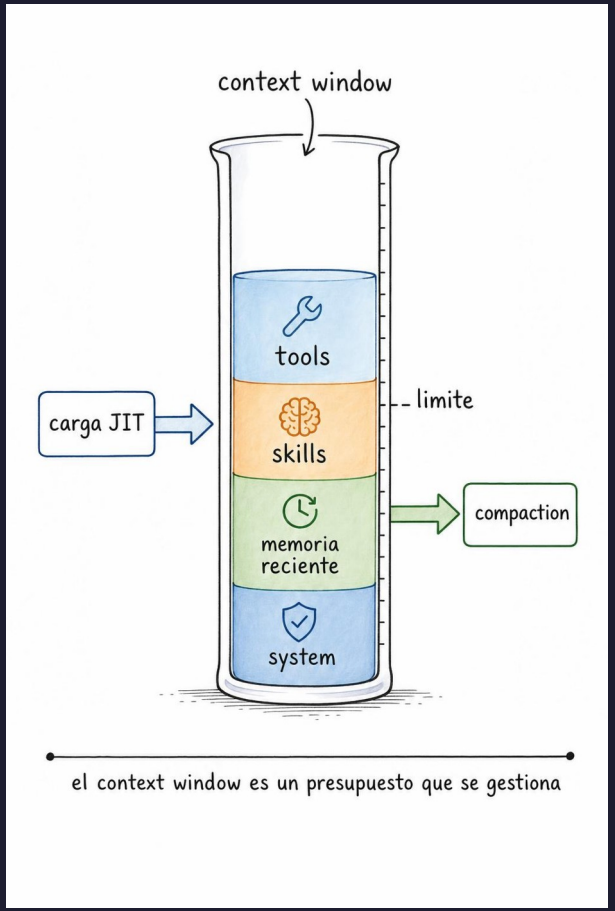
Tools · diseño ergonómico

- Las tools son la API que ve el LLM · diséñalas como una librería pública
- Names = verbos · read_file, run_shell · description = cuándo usarla
- Schema con Pydantic o JSON Schema · el LLM lee la firma, no el cuerpo
- Pocas y potentes mejor que 30+ tools específicas
- Output estructurado · JSON antes que string libre, permite chaining



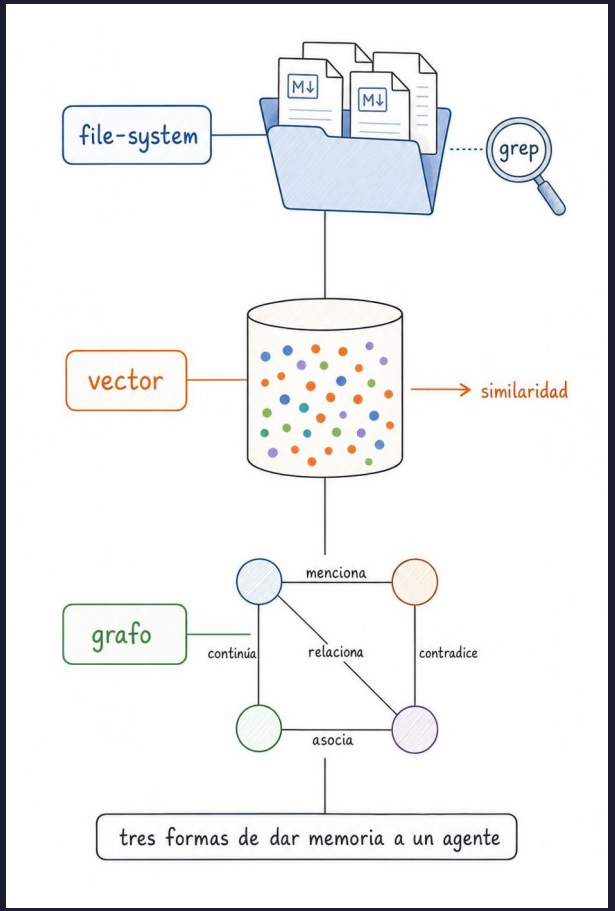
Context engineering

- El context window es un presupuesto · gestiona qué entra y qué sale
- Layered prompt · system + memoria reciente + skills + tools
- Carga JIT · solo lo necesario para el turno actual
- Compaction · resumir turnos viejos al acercarse al límite
- El context-engine es el cerebro silencioso del harness



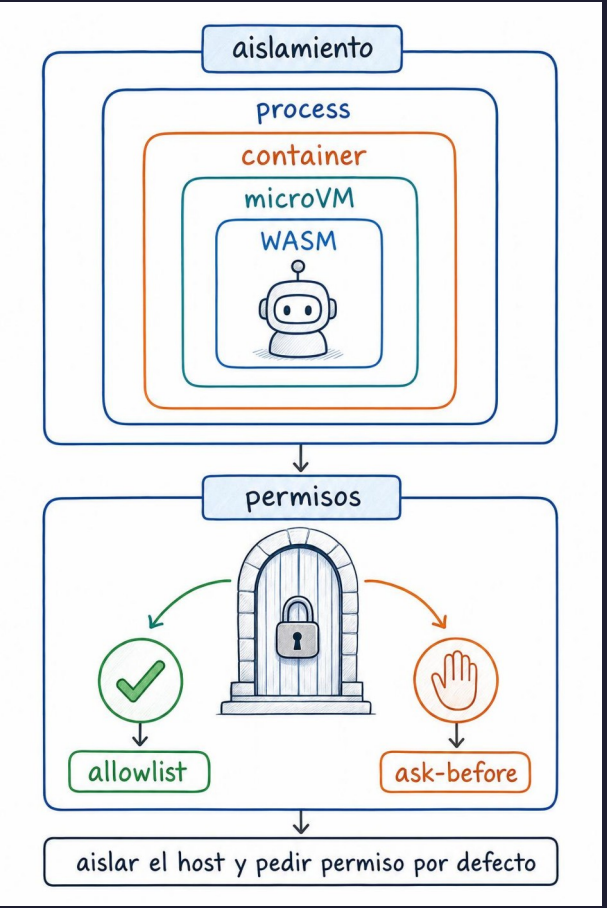
Memoria del agente

- File-system · markdown como memoria, grep como query · Claude Code
- Vector · embeddings + similarity · retrieval semántico
- Graph · KG con relaciones explícitas · razonamiento complejo
- Episódica vs semántica · qué pasó frente a qué sabemos
- Memory.md como índice · navegación rápida sin recall total



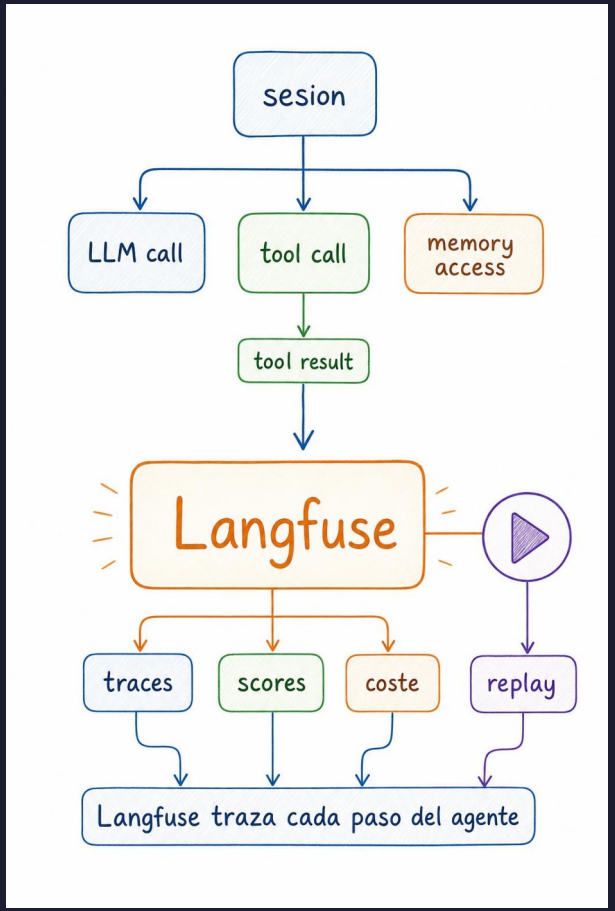
Sandboxing y permisos

- Process, container, microVM o WASM · niveles de aislamiento del host
- Allowlist mejor que denylist · safe defaults para cada tool
- Ask-before · las acciones irreversibles piden confirmación humana
- Scope y capability tokens · qué archivos puede tocar cada tool
- Sandbox por defecto siempre · y audit log de cada acción



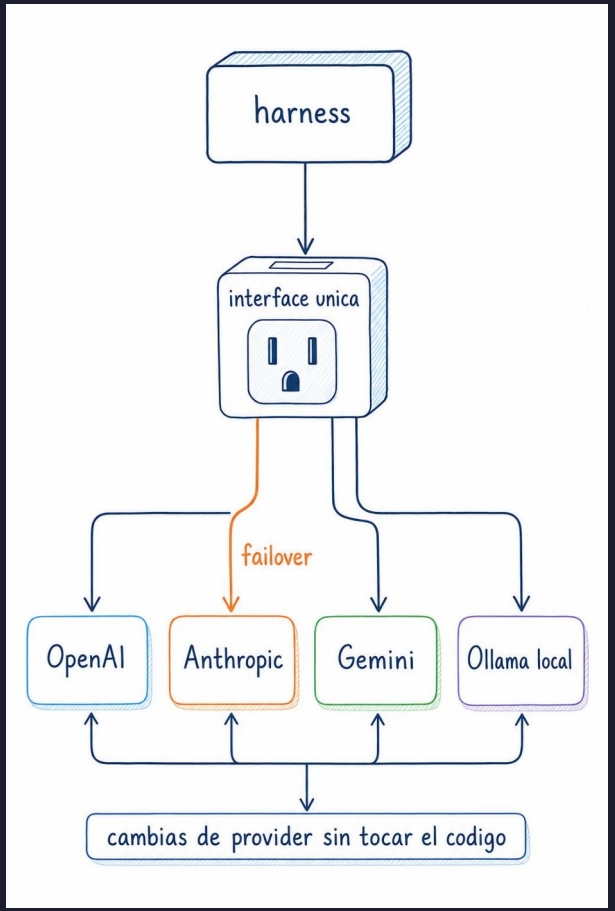
Observabilidad · traces y Langfuse

- Cada llamada LLM, tool y memory access · un span jerárquico
- Langfuse · traza cada paso del agente con coste y latencia
- Replay · reejecutar una sesión paso a paso para depurar
- Scores en Langfuse · ground truth, RAGas y evaluadores propios
- Sin observabilidad no hay depuración seria · invierte desde el día 1 (M7)



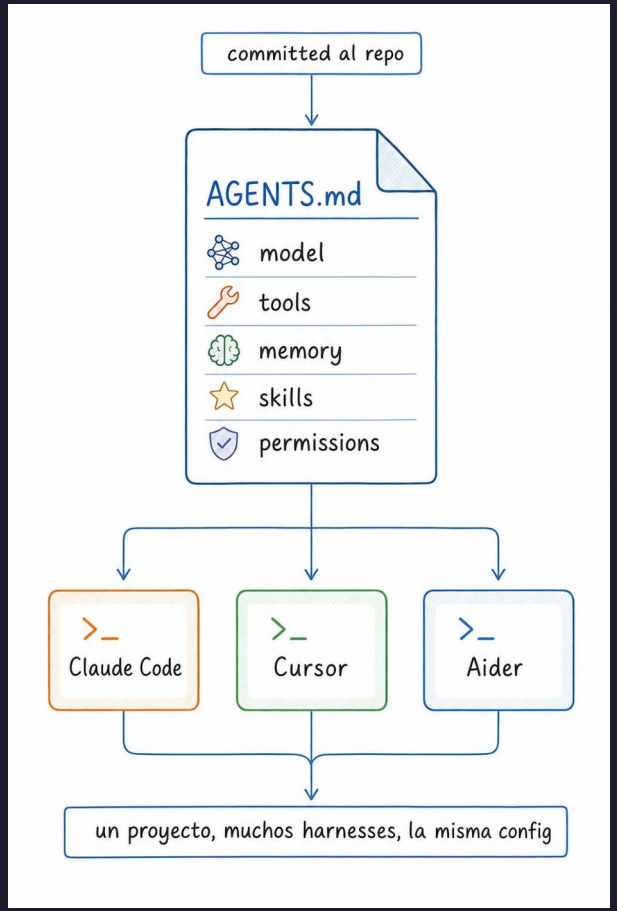
Provider abstraction · BYO model

- LiteLLM, OpenRouter, AISuite · una interfaz, varios providers
- Failover · si un provider cae, usa otro · transparente al agente
- A/B testing · reparte tráfico entre modelos sin tocar código
- LLMs locales · Ollama y llama.cpp con la misma interfaz
- El harness es agnóstico al modelo · cambias de provider, no de código



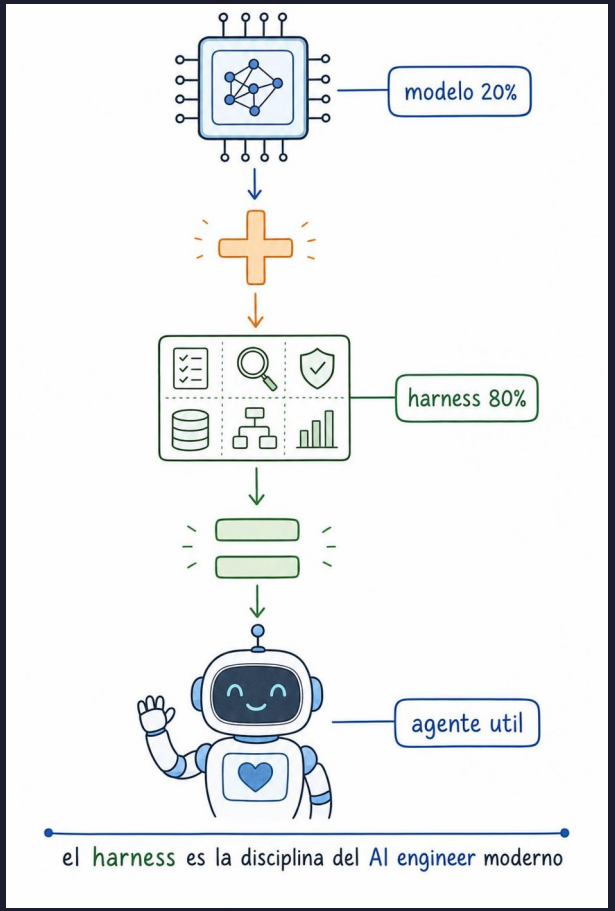
AGENTS.md · el estándar emergente

- AGENTS.md · configurar el harness por proyecto, committed al repo
- Análogo a package.json para tu agente
- Define model, tools, memoria, skills permitidos y permisos
- Adoptado por Claude Code, Cursor, Aider, OpenClaw y Codex CLI
- Un proyecto, muchos harnesses posibles · la misma config



Recap

- Harness Engineering = la disciplina del AI engineer moderno
- 6 pilares · tools, context, memory, skills, sandbox, observability
- 80% harness, 20% modelo · invierte donde mueve la aguja
- AGENTS.md · estándar para que cualquier harness ejecute tu proyecto
- M6 · cómo llevar todo esto a producción con n8n




keep coding